

```

import java.io.*;
import java.util.ArrayList;
public class GradeAnalyzer {

    public static void main(String[] args) {
        // Step 1: read scores from file
        ArrayList<Integer> scores = readScores("scores.txt");
        int invalid_lines = getInvalidLines("scores.txt");
        // Step 2: calculate statistics
        // Step pre-2: Test this method by calling it from main
with a small, hardcoded list before connecting it to the file
reader.
        //Get number of invalid lines skipped from readScores
        ArrayList<Integer> test_scores = new ArrayList<>();
        test_scores.add(100);
        test_scores.add(98);
        test_scores.add(88);
        test_scores.add(50);
        double test_average = calculateAverage(test_scores);
        System.out.println("Test average: " + test_average);
        double average = calculateAverage(scores);
        //Initialize highest to Integer.MIN_VALUE and lowest to
Integer.MAX_VALUE
        int high = Integer.MIN_VALUE;
        int low = Integer.MAX_VALUE;
        for (int score: scores){
            if(score>high){
                high = score;
            }
            if(score<low){
                low = score;
            }
        }
        ArrayList<String> grade_bands = GradeBands(scores);
        // Step 3: write and print report
        String output_file = "report.txt";
        writeReport(scores, average, high, low, grade_bands,
invalid_lines, output_file);
        printReport("report.txt");
    }

    // Returns a list of valid scores read from the file
    public static ArrayList<Integer> readScores(String filename)
{
        ArrayList<Integer> scores = new ArrayList<>();

```

```

        try (BufferedReader reader = new BufferedReader(new
FileReader(filename))) {
            String line;
            while((line = reader.readLine())!=null)
            {
                line = line.trim();
                if(line.isEmpty()){
                    continue;
                }
                try{
                    int score = Integer.parseInt(line);
                    scores.add(score);
                }
                catch(NumberFormatException e){
                    System.out.println("Warning: that is not a
number...skipping " + e.getMessage());
                    continue;
                }
            }
        } catch (IOException e) {
            System.out.println("Error reading in the file:" +
e.getMessage());
        }
        return scores;
    }
}

```

```

// Returns the number of lines skipped
public static int getInvalidLines(String filename) {
    int invalid_lines = 0;
    try (BufferedReader reader = new BufferedReader(new
FileReader(filename))) {
        String line;
        while((line = reader.readLine())!=null)
        {
            line = line.trim();
            if(line.isEmpty()){
                invalid_lines++;
                continue;
            }
            try{
                Integer.parseInt(line);
            }
            catch(NumberFormatException e){
                invalid_lines++;
                continue;
            }
        }
    }
}

```

```

    }
    } catch (IOException e) {
        System.out.println("Error reading in the file:" +
e.getMessage());
    }
    return invalid_lines;
}

```

```

// Returns the average of a list of scores, or 0.0 if the
list is empty
public static double calculateAverage(ArrayList<Integer>
scores) {
    double total = 0.0;
    if (scores.isEmpty()){
        return 0.0;
    }
    else {
        for (double score: scores){
            total+=score;
        }
    }
    double average = total/scores.size();
    return average;
}

```

```

public static ArrayList<String>
GradeBands(ArrayList<Integer> scores){
    int A = 0;
    int B = 0;
    int C = 0;
    int D = 0;
    int F = 0;
    ArrayList<String> grade_bands = new ArrayList<>();
    for(int score: scores){
        if(score>=90){
            A+=1;
        }
        else if(score>=80){
            B+=1;
        }
        else if(score>=70){
            C+=1;
        }
        else if(score>=60){
            D+=1;
        }
    }
}

```

```

        else{
            F+=1;
        }
    }
    grade_bands.add("A (90-100): " + A);
    grade_bands.add("B (80-89): " + B);
    grade_bands.add("C (70-79): " + C);
    grade_bands.add("D (60-69): " + D);
    grade_bands.add("F (below 60): " + F);
    return grade_bands;
}

```

```

        // Writes and prints the report
        /*=== Grade Analysis Report ===
Total scores processed:  13
Invalid lines skipped:   2

Average score:  77.36
Highest score:  100
Lowest score:   45

Grade distribution:
  A (90-100):  3
  B (80-89):  3
  C (70-79):  2
  D (60-69):  2
  F (below 60): 3
*/

```

```

    public static void writeReport(ArrayList<Integer> scores,
                                   double avg, int high, int
low, ArrayList<String> grade_bands, int invalid_lines,
                                   String outputFile) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(outputFile))) {
            writer.write("=== Grade Analysis Report ===");
            writer.newLine();
            writer.write("Total scores processed: " +
scores.size());
            writer.newLine();
            writer.write("Invalid lines skipped: " +
invalid_lines);
            writer.newLine();
            writer.newLine();
            writer.write(String.format("Average score:
%.2f%n", avg));

```

```

        writer.write(String.format("Highest score:
%d%n", high));
        writer.write(String.format("Lowest score: %d%n",
low));
        writer.newLine();
        writer.write("Grade distribution:");
        writer.newLine();
        for (String band: grade_bands){
            writer.write(" " + band);
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Could not write file: " +
e.getMessage());
    }
}

```

```

    public static void printReport(String reportname) {
        try (BufferedReader reader = new BufferedReader(new
FileReader(reportname))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Could not read file: " +
e.getMessage());
        }
    }
}

```

/*

Step 7: Implement writeReport

Use BufferedWriter and FileWriter to write to report.txt
Format the report neatly. Use String.format() for aligned
columns.

For example:

```

writer.write(String.format("Average score: %.2f%n", avg));
writer.write(String.format("Highest score: %d%n", high));

```

Print the same lines to the terminal using `System.out.println`.
Include the grade band counts in the report.
Your report does not need to look exactly like the example below, but it should be clearly formatted and easy to read.

=== Grade Analysis Report ===

Total scores processed: 13

Invalid lines skipped: 2

Average score: 77.36

Highest score: 100

Lowest score: 45

Grade distribution:

A (90-100): 3

B (80-89): 3

C (70-79): 2

D (60-69): 2

F (below 60): 3

Step 8: Test with Edge Cases

Test 1 (normal): Run with your original `scores.txt` and verify the report looks correct.

Test 2 (empty file): Create an empty `scores.txt` and confirm the program handles it gracefully, printing a message rather than crashing.

Test 3 (all invalid): Fill `scores.txt` with only non-number lines and confirm every line is warned and skipped.

Test 4 (single score): Put just one number in the file and confirm that all statistics are correct.

If any test causes a crash or wrong output, fix it before submitting.

Step 9: Take Your Screenshot

Make sure your terminal shows the complete program run from start to finish.

The screenshot must include:

The command used to run the program: `java GradeAnalyzer`

The full report output is printed to the terminal.

Ideally, at least one warning line showing a skipped invalid entry.

Screenshot methods

On Windows: use `Win+Shift+S` to capture part of your screen.

On Mac: use `Cmd+Shift+4` to select an area.

On Linux: use the screenshot tool for your desktop environment.

Step 10: Convert and Submit
In the textbox below:

Upload your GradeAnalyzer.java file as a PDF.
Copy into a document and export to PDF
Upload your scores.txt file as a PDF.
File > Print > Save as PDF
Upload your terminal screenshot as a PDF.
If your environment is not working and you cannot reach this point, post in Piazza right away. Do not wait. Your cohort and the course team are there to help.
*/